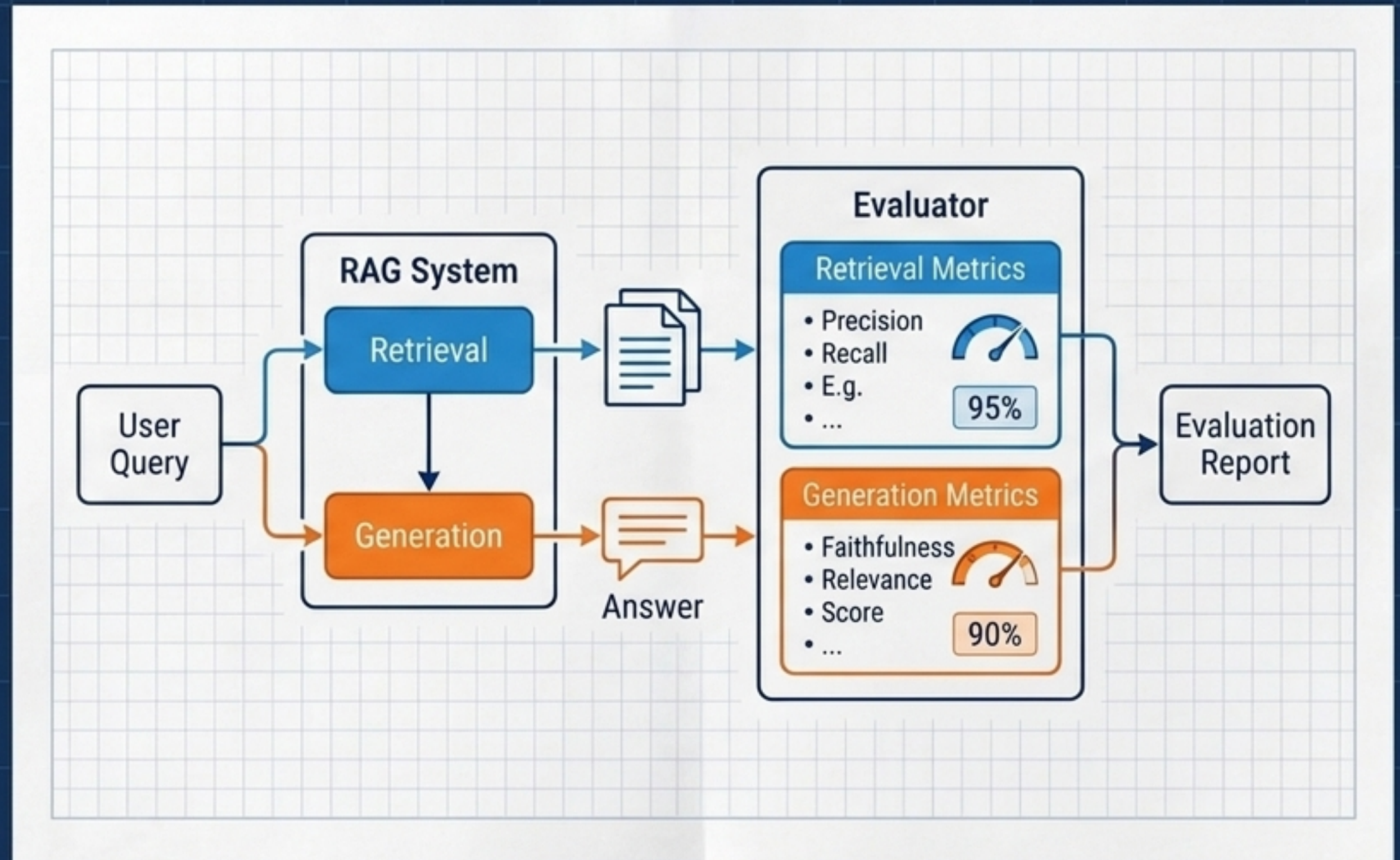


RAG Evaluator Project

Diagnosing Retrieval and Generation Pipelines through Independent Evaluation



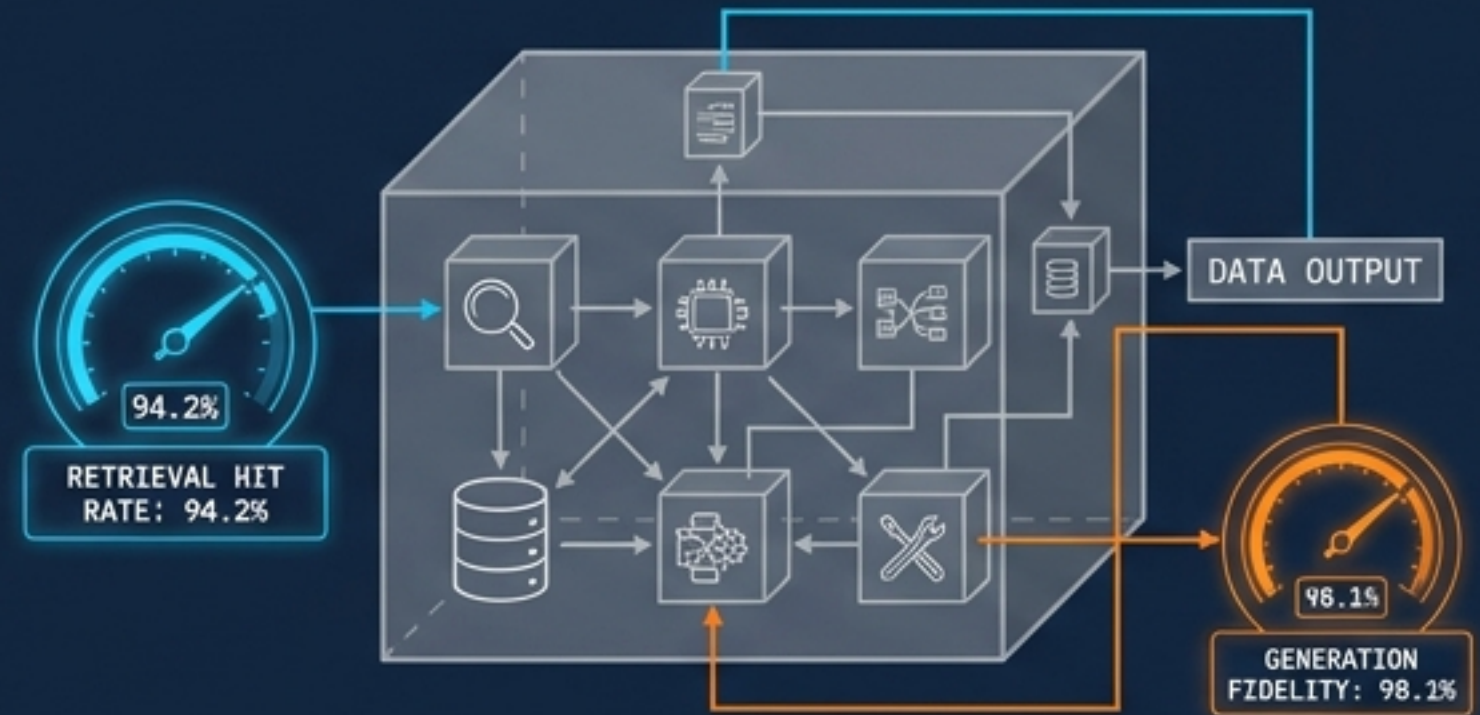
An unevaluated RAG system is not a production system.

The Prototype



Unknown hit rates, **silent** hallucination failures.

The Production System



Continuous telemetry measuring **exact retrieval accuracy** and **generation fidelity**.

To move from prototype to production, we must build a **diagnostic instrument** that measures **whether the pipeline is actually working**—and if it is not, **exactly which part is failing**.

Diagnosis requires separating retrieval from generation.

eval_retrieval.py

Document

Vector DB

Did we find the right context?

Known-Good
Context

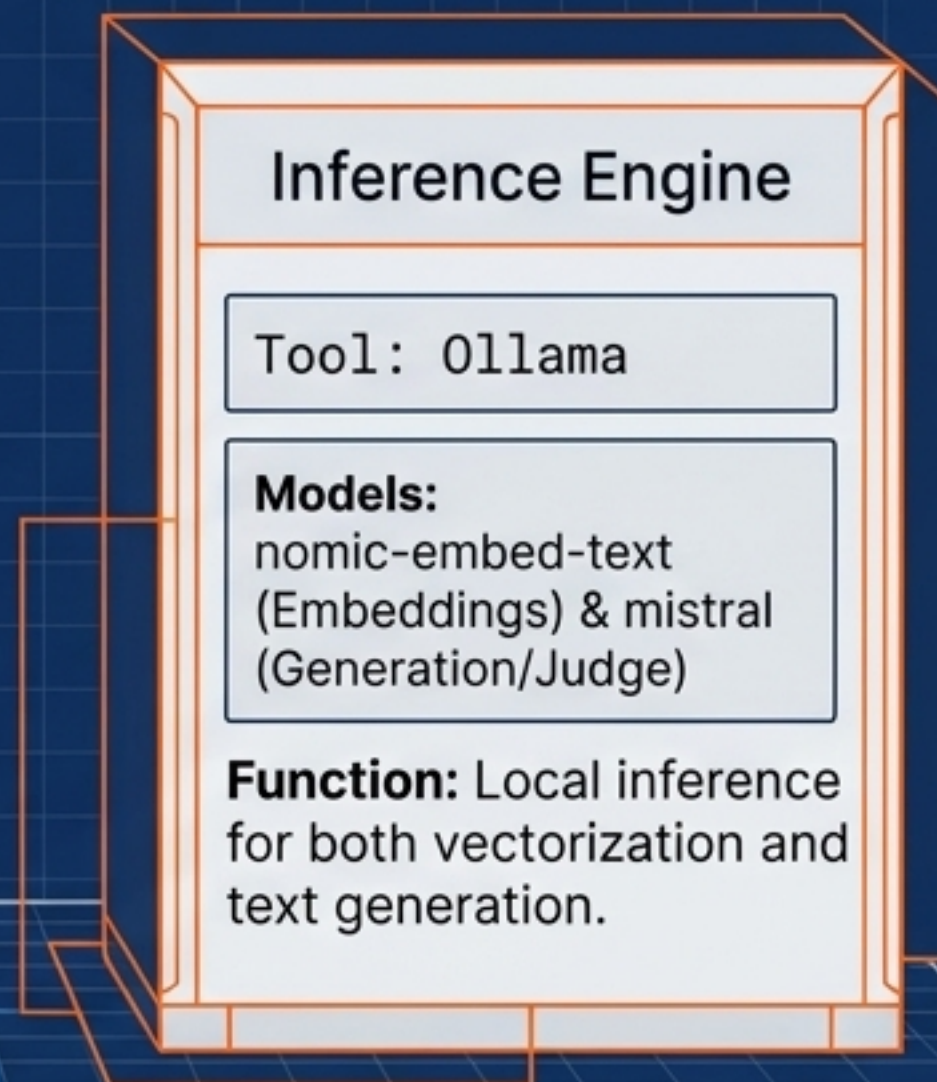
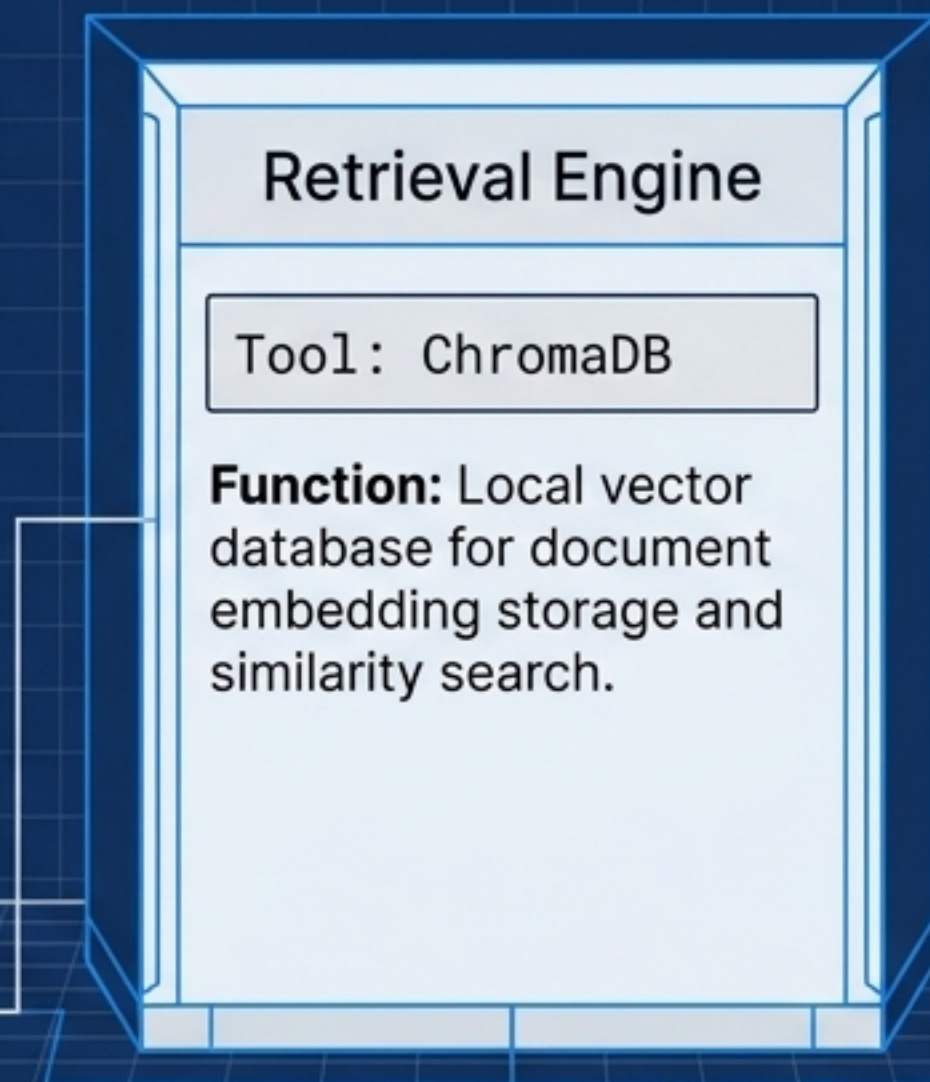
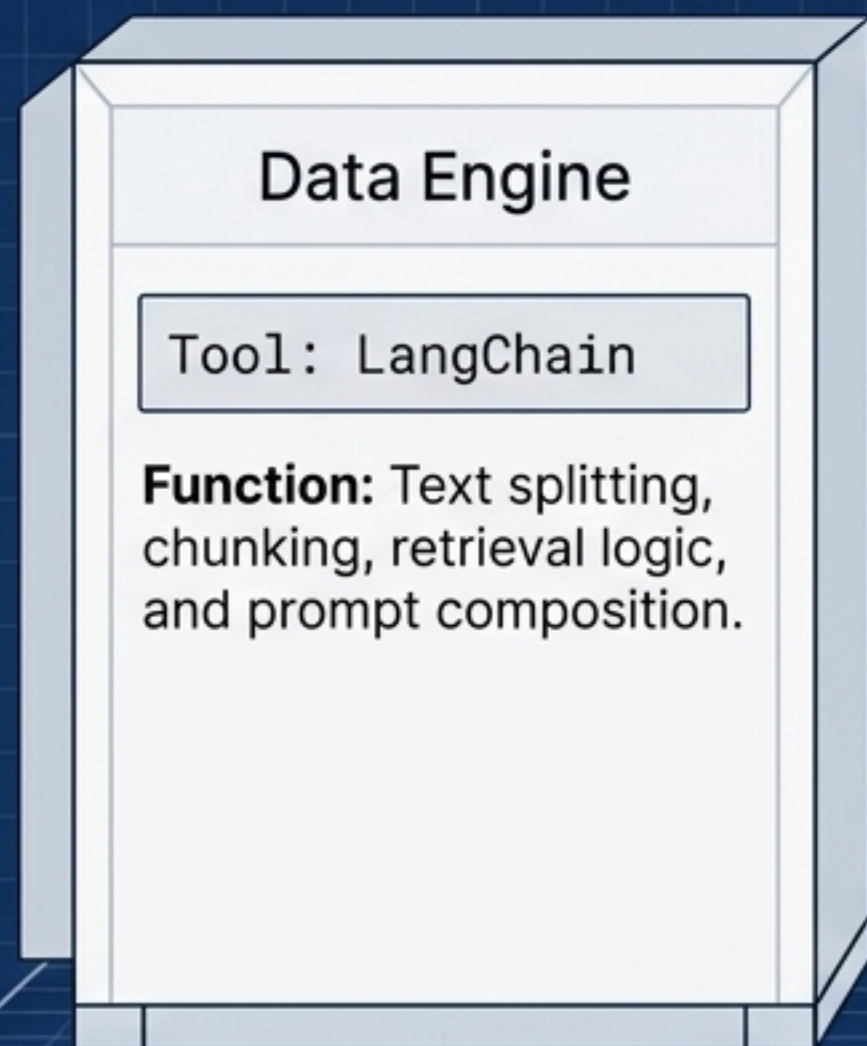
eval_generation.py

Prompt

LLM

Given perfect context, can the LLM
synthesize the right answer?

A fully local stack enables rapid, private evaluation loops.



A structured Golden Test Set anchors the evaluation



15 Passages: SQuAD v1.1
(General knowledge/Science/History)



10 Manual Pairs: Telecom Domain
(Specific, domain-relevant Q&A)



data/golden_test_set_squad.csv

1	question	(string)
2	expected_source	(string)
3	expected_answer	(string)
4	source_type	(enum: squad/telecom)

Two independent evaluators measure distinct failure modes.

Retrieval Test (eval_retrieval.py)

Target Component: Vector DB & Embeddings

Input: Question + Expected Source ID

Core Metric: Hit@K, Mean Reciprocal Rank (MRR)

LLM Involvement: None (pure math/similarity search)

Key Question: Are we retrieving the correct document?

Generation Test (eval_generation.py)

Target Component: LLM Prompting & Inference

Input: Expected Answer + Expected Source Text (bypassing DB)

Core Metric: Generation Pass Rate (%)

LLM Involvement: High (LLM-as-a-judge using Mistral)

Key Question: Can the model extract the answer from perfect context?

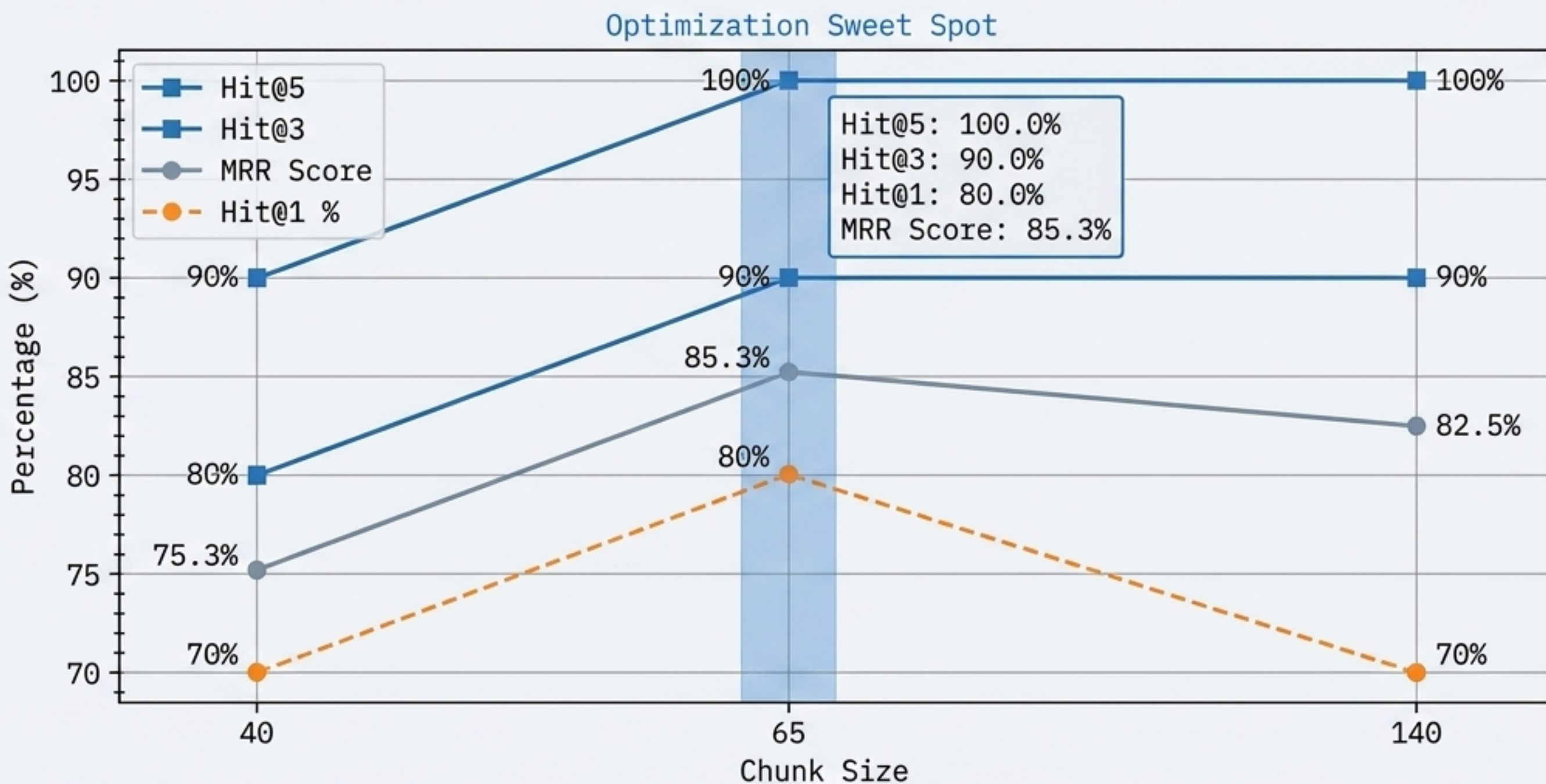
Mean Reciprocal Rank measures precision, not just presence.



$$\text{MRR} = \frac{1}{\text{number of queries}} \times \sum_i^n \frac{1}{\text{rank of first correct result}}$$

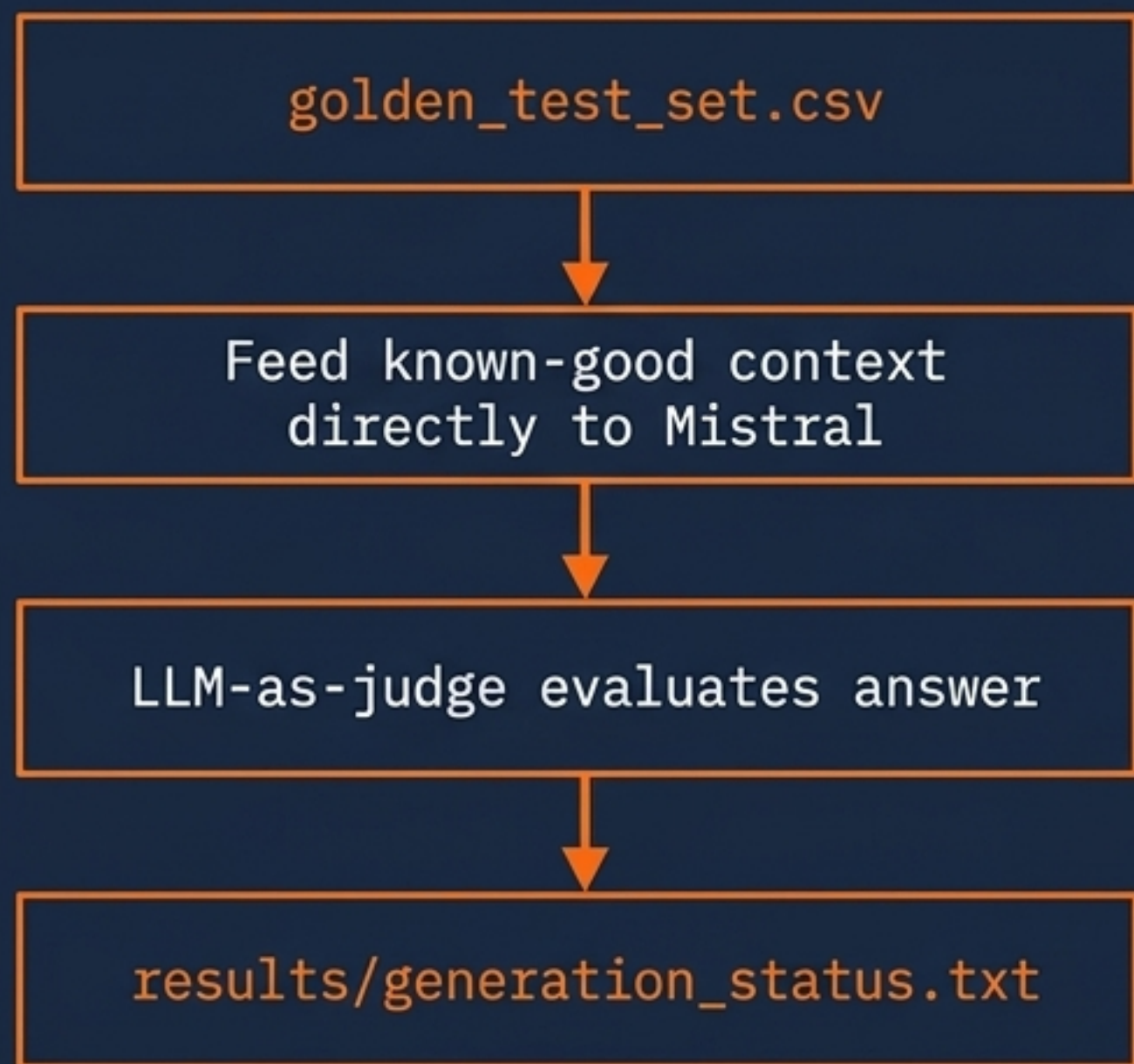
Takeaway: Hit@K only verifies if the right chunk appeared. MRR verifies how high up in the context window it was placed, minimizing LLM distraction.

Chunk size 65 yields the optimal retrieval balance.



While size 140 maintains perfect Hit@5, Hit@1 degrades from 80% to 70%. Size 65 forces tighter embedding matches, ensuring the best context ranks first.

Bypassing retrieval reveals a generation bottleneck.



68.0%

GENERATION PASS RATE

Even when given the perfect, hand-selected context paragraphs (completely bypassing ChromaDB), the Mistral model only successfully extracted the correct answer 68% of the time.

System Diagnosis: Perfect memory, flawed articulation.



Retrieval Health

100%

(Hit@5 at chunk size 65)

[OPTIMIZED]

The vector database is flawlessly retrieving the necessary documents.



Generation Health

68%

(Pass Rate)

[CRITICAL BOTTLENECK]

The LLM is failing to utilize the perfect context provided to it.

Synthesis Conclusion

Because we isolated our evaluation, we know exactly where to spend our engineering effort. Tuning embeddings or chunk sizes will yield zero ROI. We must redesign the LLM prompt.

Diagnostic telemetry maps symptoms directly to fixes.

Symptom	Root Cause	Prescribed Fix
Low Hit@K, Low MRR.	The database cannot find the documents.	Adjust chunk overlap, try semantic chunking, or upgrade embedding model (nomic-embed-text -> bge-m3).
High Hit@K, Low MRR.	Documents are found, but buried at the bottom of the context window.	Implement a re-ranking layer before passing context to the LLM.
High Hit@K/MRR, Low Gen Pass Rate (Our Current State).	LLM is ignoring or misunderstanding the retrieved context.	Re-engineer the system prompt, enforce output formatting, or switch LLM.

Actionable telemetry transforms prototypes into systems.

The Final Law of RAG: A golden test set and isolated evaluators turn subjective LLM vibes into deterministic engineering metrics.

Next Steps for this Project:

1. Implement advanced prompt engineering to fix the **68% generation bottleneck**.
2. Introduce query rewriting/paraphrasing (query_data.py).
3. Automate evaluation logging in CI/CD pipeline.

EVALUATION PIPELINE:
COMPLETE.
STATUS: READY FOR
OPTIMIZATION.